

WebSpec: Towards Machine-Checked Analysis of Browser Security Mechanisms

Lorenzo Veronese, Benjamin Farinier, Mauro Tempesta, Marco Squarcina, Matteo Maffei
TU Wien

Abstract—The complexity of browsers has steadily increased over the years, driven by the continuous introduction and update of Web platform components, such as novel Web APIs and security mechanisms. Their specifications are manually reviewed by experts to identify potential security issues. However, this process has proved to be error-prone due to the extensiveness of modern browser specifications and the interplay between new and existing Web platform components. To tackle this problem, we developed WebSpec, the first formal security framework for the analysis of browser security mechanisms, which enables both the automatic discovery of logical flaws and the development of machine-checked security proofs. WebSpec, in particular, includes a comprehensive semantic model of the browser in the Coq proof assistant, a formalization in this model of ten Web security invariants, and a compiler turning the Coq model and the Web invariants into SMT-lib formulas.

We showcase the effectiveness of WebSpec by discovering two new logical flaws caused by the interaction of different browser mechanisms and by identifying three previously discovered logical flaws in the current Web platform, as well as five in old versions. Finally, we show how WebSpec can aid the verification of our proposed changes to amend the reported inconsistencies affecting the current Web platform.

Note. This is a preprint of a paper submitted to IEEE S&P '22 on 3 Dec 2021.

I. INTRODUCTION

Web browsers are considered among the most complex software in use today, and the number of Web platform components, i.e., browser functionalities and security mechanisms, is constantly increasing. These are typically proposed by browser vendors in the form of a W3C *Editor's Draft* and discussed within the community. If enough consensus is reached, the standardization process has to progress through several *maturity levels* before becoming a W3C recommendation.

While the implementation of new Web platform components is subject to extensive compliance testing (see, e.g., the *Web Platform Tests* project [6]), their specifications undergo a manual expert review to identify potential issues: this is a continuous and extremely complex process that has to consider the interplay with legacy APIs and should, in principle, be revised whenever new components land on the Web platform.

Unfortunately, manual reviews tend to overlook logical flaws, eventually leading to critical security vulnerabilities. For example, the `HttpOnly` flag was introduced by Internet Explorer 6 [18] as a way to protect the confidentiality of cookies with this attribute by not exposing them to scripts. Eight years after its launch, Singh et al. discovered that this property could be trivially violated by any scripts accessing the response

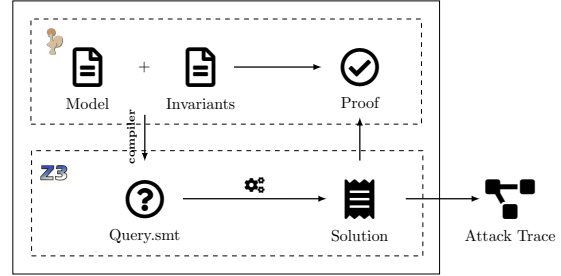


Fig. 1. The WebSpec framework.

headers of an AJAX request via the `getResponseHeader` function [38]. Security vulnerabilities at the level of Web specifications have also affected CORS [8], CSP [39], and Trusted Types [5], to name a few.

We argue that this dire situation stems from several concurring factors: (i) Web platform components are specified informally and therefore their analysis, albeit conducted by expert eyes, may easily overlook corner cases; (ii) there is no precise understanding of which security properties should be seen as invariants in the Web and, thus, be preserved by updates of the Web platform; (iii) Web platform components are typically evaluated in isolation, without taking into account their interactions, that is, the entangled nature of the Web platform.

Our Contributions: In this work, we advocate a paradigm shift, letting Web platform components and their interplay undergo a formal security analysis as opposed to a manual expert review. In particular, we introduce WebSpec, the first formal framework for the security analysis of browser security mechanisms that supports the automated detection of logical flaws as well as machine-checked security proofs. As outlined in Figure 1, WebSpec includes:

- a formal browser model in Coq, which formalizes a core set of Web platform components, including well-established components (cookies, SOP, CORS, etc.) and recently introduced ones (e.g., CSP level 3 and Trusted Types);
- the formal definition of ten invariants that are expected to hold in the Web (e.g., the integrity of `__Host-` cookies and the fact that a page protected by CSP can only be read/modified by the scripts allowed by the policy);
- a compiler, which translates the browser model and

the invariants into SMT-lib formulas in order to enable bounded model checking by the Z3 automated theorem prover. In case a violation is found, WebSpec reconstructs the minimal sequence of actions leading to it, visually displaying the corresponding attack.

We demonstrate the effectiveness of WebSpec by:

- discovering a new attack on cookies caused by the interaction with legacy APIs and a new inconsistency between CSP and a planned change to the HTML standard;
- rediscovering three previously reported logical flaws in the current Web platform;
- adjusting the model to reflect past states of the Web platform in order to identify five previously published attacks, with the goal of showing that automated security analysis would have prevented these vulnerabilities;
- conducting four proofs in the Coq model, showing the correctness of our proposed changes to fix vulnerabilities that are currently affecting the Web platform, including a new technique against a Trusted Types bypass.

Outline: Section II introduces our browser model. Section III presents the compiler turning the browser semantics and the Web invariants into SMT-lib formulas. Section IV illustrates the formal definition of the aforementioned Web invariants along with several inconsistencies identified by WebSpec. Section V reports on machine-checked security proofs conducted within our browser model. Section VI reports on the performance of the compiler and discusses further optimizations. Section VII discusses related works and Section VIII concludes.

II. BROWSER MODEL

This section provides an overview of the main components of our browser model written in Coq. The model focuses on Web platform components, i.e., browser functionalities and security mechanisms, abstracting away from the network and Web servers. Our formalization enables reasoning about all possible sequences of events leading to an inconsistent state without necessarily having to model a specific Web application. We are indeed interested in proving Web invariants that should hold for all Web applications, irrespectively of application-specific assumptions that attackers could violate. For instance, scripts in our model can, in principle, execute arbitrary sequences of any of the API calls we support, as this would be the case in the presence of cross-site scripting attacks. The model also includes configuration flags that enable reasoning on former states of the Web platform or testing new proposals prior to their implementation.

A. Core Abstractions

The browser is modeled as a transition system in which a state evolves from an initial to a final configuration following a list of events and according to an inductive relation named *Reachable* parameterized by a global environment:

Inductive *Reachable* : Global $\rightarrow$ list Event $\rightarrow$ State $\rightarrow$ Prop.

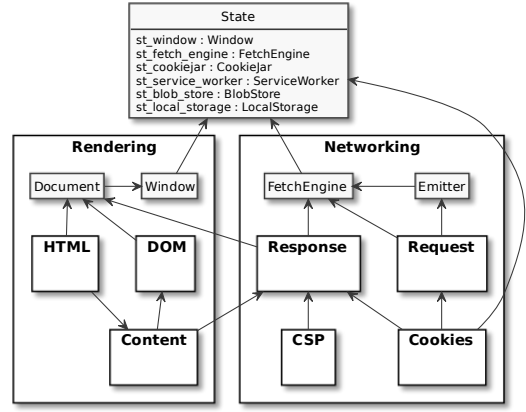


Fig. 2. Browser State and overview of its components

Intuitively, given a global environment *gb*, a list of events *evs*, and a state *st*, *Reachable gb evs st* means that, starting from a given initial state, *st* is reachable by executing sequentially the events in *evs* under environment *gb*.

The Global environment contains concrete values (e.g., the Web browser configuration) or symbolic variables (e.g., a set of pages) which are constant through the evolution of the browser state. An Event represents an atomic action that modifies the state, e.g., sending a network request or updating the DOM, which may originate from different sources, such as the browser itself, a script, or a service worker. A State is a collection of datatypes used to model browser components. A detailed overview of the components supported by our model is provided in the following sections.

Based on these ingredients, we formalize invariants within our model as follows, where *hypothesis* and *conclusion* are predicates that may refer to the global environment, past events, or the current state of the browser:

```

1 Parameter hypothesis : Global  $\rightarrow$  list Event  $\rightarrow$  State  $\rightarrow$  Prop.
2 Parameter conclusion : Global  $\rightarrow$  list Event  $\rightarrow$  State  $\rightarrow$  Prop.
3
4 Definition Invariant (gb : Global) (evs : list Event) (st : State)
  := Reachable gb evs st  $\rightarrow$  hypothesis gb evs st  $\rightarrow$ 
  conclusion gb evs st.

```

We present instances of Web invariants in Section IV.

B. Page Rendering

The main component used to model the rendering functionality is the Window datatype. A Window in our model represents a window in terms of browsing context [52, §7.1], i.e., an environment in which the browser displays a document. The field *wd_location* is the URL being visited and *wd_document* contains the displayed document. Since a Window can represent either a top-level window or a frame, *wd_parent* contains an optional index which, if empty, denotes a top-level window or points to the parent frame otherwise. Similarly, *wd_initiator* contains an optional index which is used to track the source browsing context of this window [52, §7.11] by storing a reference to the window responsible for starting the navigation.

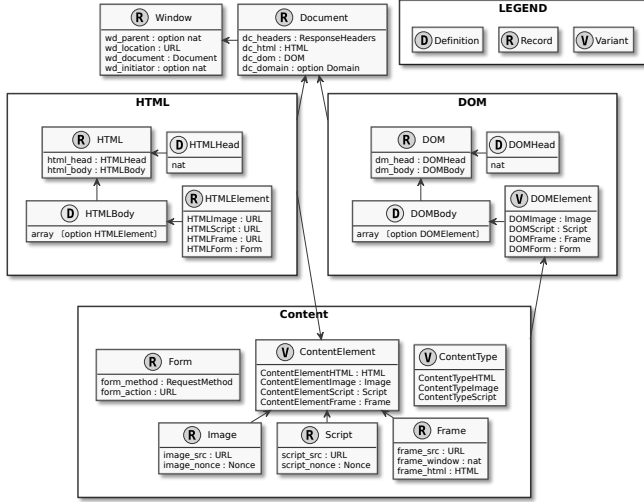


Fig. 3. Page rendering components

A `Document` represents a Web page loaded and rendered in a browser window. When a page is loaded, `dc_html` represents the HTML code of the response, while `dc_dom` contains the rendered elements of the page. Static elements, e.g., forms, and possibly other markup tags, are rendered immediately. Subresources of the page, such as frames and scripts, require an additional request to be included in `dc_dom`. For instance, the presence of a `HTMLFrame` in `dc_html` might cause three additional events to be executed in sequence: a request (`EvRequest`), a response (`EvResponse`), followed by the update of the DOM (`EvDOMUpdate`) resulting in `DOMFrame` being added to `dc_dom`. This approach enables fine-grained modeling of the rendering process of the browser. In particular, our model can capture the order in which resources are loaded and the presence or absence of specific elements.

WebSpec currently supports forms with the `method` and `action` attributes, images, scripts, and frames. Rendered frames in `dc_dom` contain a reference to the corresponding Window (cf. Figure 3), reflecting the tree-like structure of the DOM. Finally, `Document` also includes the list of headers (`dc_headers`) in the HTTP response used to render the page and `dc_domain`, an optional field used to model domain relaxation via the `document.domain` API.

C. Networking and Cookies

The main component used to model the networking functionality is the `FetchEngine`, which abstracts network access and is responsible for sending requests and receiving responses. `ft_request` contains the last emitted request, and `ft_emitter` maps to the originator of the request, i.e., whether the request is top-level or generated by the inclusion of subresources, issued by a script, a form, a worker, or it is a CORS preflight. `ft_response` is a field that either contains the corresponding response or is empty if the request is still pending. Finally, we store `Emitter * Request *`

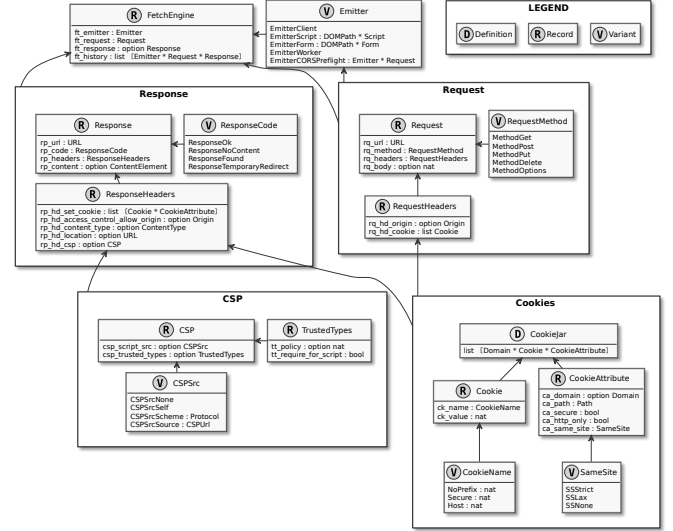


Fig. 4. Networking and cookies components

`Response` triples in `ft_history` in order to keep track of previous network accesses.

The modeling of `Request` and `Response` is rather straightforward, as shown in Figure 4. We support requests and responses through HTTP and HTTPS protocols. For requests, we model the HTTP methods GET, POST, PUT, DELETE, and OPTION. Concerning responses, the following HTTP status codes are supported: (i) 200 OK, successful response, (ii) 204 No Content, successful response with an empty body, (iii) 302 Found, redirection with no integrity guarantees in the redirected request over the HTTP method and the body of the original request [35], (iv) 307 Temporary Redirect, redirection enforcing that the method and body of the original request are preserved in the redirected one.

Supported headers are `Origin`, `Cookie` and `Referer` for requests, and `Content-Type`, `Set-Cookie`, `Location` and `Referrer-Policy` for responses. We also include `Content-Security-Policy` and `Access-Control-Allow-Origin` to support CSP [45] and CORS [48], respectively. Currently modeled CSP directives are: `script-src` that defines the allowlist for JavaScript sources, and `trusted-types` together with `require-trusted-types-for` for Trusted Types [30] support, as explained in Section II-D.

Cookies are stored in the `CookieJar` as a list of triples in the form `Domain * Cookie * CookieAttribute`, where `Domain` represents the host setting the cookie, `Cookie` is a pair corresponding to the name of the cookie and its value, and `CookieAttribute` is a record containing the attributes. Currently modeled cookie attributes are `Domain`, `Path`, `Secure`, `HttpOnly`, and `SameSite`. We also support the `__Secure-` and `__Host-` naming prefixes [50].

D. Additional Features

Starting from the core functionalities discussed in the previous sections, our model can be extended to support other Web components, including novel security mechanisms that could benefit from the automated formal analysis enabled by WebSpec. We discuss in the following five additional Web components supported by our model.

Service Workers and Cache API. Service workers [49] act as proxy servers that sit between Web applications, the browser, and the network. We reflect this capability in our model by considering a specific kind of service worker that can perform fetch requests, serve synthetic responses, and cache pairs of requests and responses, regardless of the scope. To this end, we also model a lightweight Cache API and assume that Service Workers have arbitrary access to it.

Local URLs. We model requests to local URLs [51], i.e., URLs with a local scheme such as `data:` and `blob:`, as virtual requests that do not generate a response from the network. We partially support the File API [46] by enabling the creation of blob URLs via the `URL.createObjectURL` JavaScript method. We assume that local URLs are accepted interchangeably with remote URLs, meaning that they can be navigated by frames, or included as a script in a page.

Local Storage. The Web Storage API [52, §12] enables JavaScript to store and retrieve key/value pairs in the browser. The API provides two mechanisms to store data: `sessionStorage`, an ephemeral storage that expires when the browser or the page is closed, and `localStorage`, which persists in the browser unless cleared explicitly. As we are interested in capturing single browser sessions, the difference between the two mechanisms is irrelevant. For this reason, we model only `localStorage`, providing methods to read and write data in the browser storage from any scripts.

Web Messaging. Cross-origin communication is enabled by the `postMessage` API [52]. As we are interested in modeling messages that are sent and received—while we ignore messages that do not reach destination—we encode the sending and receiving of a message as a single action. We also model the origin validation process performed in the receiving script. In this way, we can capture potential security issues due to cross-origin messages processed without validating the origin of the sender [42].

Trusted Types. Trusted Types are a novel security mechanism designed to prevent DOM XSS by guarding injection sinks with application-defined policies [30]. We model the enforcement of Trusted Types on a page by mandating scripts to invoke the Trusted Type API to create a `TrustedHTML` object, and use it to modify the DOM via the `Element.innerHTML` property. Although we do not model the content of policies, we encode the ability to disallow the creation of any Trusted Types via the CSP directive `trusted-types 'none'`.

E. JavaScript

Contrary to previous works [15], instead of modeling scripts with an internal state and precise small-step semantics, we

model them in terms of actions that the browser can perform. Since we are not interested in application-specific behavior, our abstraction captures the execution of sequences of Web API calls and the evolution of the browser’s state. For example, we model the fact that a script can set a cookie, or add a request-response pair to a cache using the Cache API, but we do not model how cookie data, requests, or responses are built. Instead, we introduce symbolic variables with constraints following the API specification.

Scripts in our model can update the DOM, set and get cookies using `Document.cookie`, or navigate frames using the `Window.location` setter. They can use the Fetch API to perform network requests and read the corresponding responses (up to SOP constraints). We support the restrictions imposed by the SOP and model relaxation mechanisms such as CORS and the legacy `Document.domain` API.

Scripts can also use the APIs described in Section II-D: they can update a cache from page context using `Cache.add` or `Cache.put`, communicate with other windows using `Window.postMessage`, perform domain relaxation by setting `Document.domain`, create blob URLs with `URL.createObjectURL`, and create Trusted Types objects.

III. COMPILATION AND TRACE RECONSTRUCTION

In the following, we present our compiler which translates the browser model and the Web invariants written in Coq into a query that can be automatically checked by an SMT solver – Z3 in the current implementation (cf. Figure 1). We then explain how to reconstruct an attack trace, which enjoys correctness and minimality, from a SMT solution.

A. From Coq to a SMT Query

For every Web invariant that we aim to verify in our model, we define a corresponding query (as a Coq inductive type) that is satisfied if a counterexample to the invariant is found in any of the states reachable from the initial browser state. Essentially, queries are Coq types of the following form:

```
1 Inductive Query (gb: Global): list Event → State → Prop :=
2 | CounterExample :
3   ∀ evs st,
4     Reachable gb evs st →
5     hypothesis gb evs st →
6     not (conclusion gb evs st) →
7     Query gb evs st.
```

To automatically verify the (in)validity of an invariant, we developed a compiler that translates the Coq model and the corresponding query into SMT-lib formulas, which are then fed to the Z3 solver. Technically, we compile Coq inductive types into CHC logic, i.e., first-order logic with fixed-points expressed in terms of Constrained Horn Clauses [28], [27], in order to find inhabitants of the translated inductive types, i.e., terms of these types. In particular, the compiler translates `Query`, `Reachable` and all the inductive types of kind `Prop` involved in the definitions of `hypothesis` and `conclusion` into relations expressed in terms of Horn clauses, while the remaining inductive types, including the browser state, the list of events and the global environment, are instead translated into SMT datatypes.

We refer the interested reader to Appendix B for a discussion of the fragment of the Coq logic supported by our compiler and for further details about the compilation pipeline.

B. From a SMT Solution to an Attack Trace

We use μZ , a Z3 extension with fixed-point [28], [27], to solve the SMT-formula produced by our compiler. The four possible outcomes are the following:

- 1) μZ finds a solution, hence the invariant does not hold. We discuss in Section IV the security implications of violating an invariant.
- 2) μZ fails to find a solution or to prove its absence. In such a case, which never happened in our case studies, we cannot draw any conclusion.
- 3) μZ proves that there is no solution. Since neither our compiler nor μZ are formally verified, we cannot directly conclude that the invariant holds. However, this gives us strong confidence that this is the case and it is worth starting a formal proof in Coq, as shown in Section V.
- 4) μZ does not terminate. Due to the way μZ works, this means that μZ did not find a solution after exploring a certain number of steps. When this number becomes high enough, though it is not a proof, it gives us a good intuition that the invariant holds. As in the previous case, a formal proof can be manually produced, if needed.

When μZ finds a solution (case 1), we first verify its correctness by automatically translating it back into a Gallina expression (the language of Coq) and checking whether the resulting term is an inhabitant of `Query`. Since μZ instantiates all symbolic variables, the proof is straightforward and mostly automatic. This step is needed because, as mentioned previously, we do not yet have a correctness proof of the compiler.

The following theorem formalizes that inhabitants of `Query` are indeed counterexamples of the invariant and concludes the correctness proof of the solution:

```
1 Theorem counterexample_invalidate_invariant :
2   ∀ gb evs st, Query gb evs st → not (Invariant gb evs st).
3 Proof.
4   intros gb evs st Q I.
5   unfold Invariant in I.
6   destruct Q as (evs,st,R,H,C).
7   specialize (I R H).
8   congruence.
9 Qed.
```

As the last step, we automatically extract an attack trace from the solution provided by μZ . It is worth noting that, besides proving that the generated trace is indeed a counterexample of the invariant, the attack trace has the property of being minimal. This property is due to the resolution algorithm implemented in μZ , which ensures that the list of events in the solution is the smallest one that leads to a counterexample. WebSpec then renders this trace as a sequence diagram, making the representation of the counterexample accessible to users not familiar with formal verification. Examples of such diagrams are given in Section IV.

TABLE I
WEB INVARIANTS

Web Feature	Invariant	Holds	References
Cookies	Confidentiality of <code>Secure</code> cookies (network)	●	[12]
	Confidentiality of <code>HttpOnly</code> cookies (Web)	●	[38], [12]
	Integrity of <code>__Host-</code> cookies	○	[38], [50]
CSP	Interaction with SOP	○	[45], [39]
	Integrity of server-provided policies	○	[40]
	Access control on Trusted Types DOM sinks	●	[30]
	Safe policy inheritance	●	[45]
Origin Header	Authenticity of request initiator	●	[8], [48]
SOP/CORS	Authorization of non-simple requests (<i>i</i>)	●	[8], [47]
	Authorization of non-simple requests (<i>ii</i>)	●	[8], [52]

IV. WEB INVARIANTS

Web specifications might make invalid assumptions about the interplay of different components in the Web platform and thus introduce inconsistencies that can impact the security of Web applications. Intuitively, a Web invariant is a *property of the Web platform that is expected to hold across its updates and independently on how its components can possibly interact with each other* [8]. In this work, we define 10 Web invariants concerning 5 core Web components (see Table I). First, we exemplify the encoding of Web invariants in our model and show how to derive a query from the invariant using a well-known property of the `Secure` cookie attribute. Then we discuss a selection of invariants that do not hold in the current Web platform. In particular, we show how WebSpec is able to discover a new attack on the `__Host-` prefix for cookies as well as a new inconsistency between the inheritance rules for the Content Security Policy and a planned change in the HTML standard. Due to space constraints, we refer to Section A for the invariants that have already been discussed in the literature and now hold in the current Web platform.

A. Cookies

Cookies are the main state management mechanism of the Web, allowing servers to maintain a stateful session over the stateless HTTP protocol [12]. Servers can store state in the browser through the `Set-Cookie` header. This state is later on automatically attached by the browser to all following HTTP requests to the server via the `Cookie` header. Cookies can be configured with attributes (or flags) which specify, for instance, that a cookie cannot be sent over insecure channels (`Secure`), or that a cookie must not be accessible from JavaScript code (`HttpOnly`). Cookie prefixes [50] allow Web applications to specify additional properties for cookies by remaining backward compatible. The specification defines two prefixes that enforce additional constraints on cookies in compatible browsers: (i) when a cookie name begins with `__Secure-`, the cookie must be set with the `Secure` attribute and from a page served over HTTPS; (ii) when a cookie name begins with `__Host-`, the cookie has all the constraints of the `__Secure-` attribute, plus the `Path` attribute must be set to the value `"/"` (ensuring that the cookie will be attached to all requests) and must not contain a `Domain` attribute, thus restricting the scope of the cookie to the host that set it.

1) *Confidentiality of Secure Cookies*: Starting from version 52, both Chrome and Firefox prevent insecure sites (loaded via HTTP) from setting Secure cookies. This corresponds to the following invariant.

Invariant. *Cookies with the Secure attribute can only be set (using the Set-Cookie header) over secure channels.*

This invariant is encoded in WebSpec as the following Coq definition:

```
1 Definition SecureCookiesInvariant (gb: Global) (evs: list
  Event) (st: State): Prop :=
2   ∀ rp corr _evs cookie,
3   Reachable gb evs st →
4   evs = (EvResponse rp corr :: _evs) →
5   rp_hd_set_cookie (rp_headers rp) = Some cookie →
6   sc_secure cookie = true →
7   url_protocol (rp_url rp) = ProtocolHTTPS.
```

Intuitively, this definition says that for every reachable state where the browser is handling a network response, i.e., the state is `Reachable` and the current event is `EvResponse` (lines 2-4), if the response contains a `Set-Cookie` header (line 5) with a cookie that has the `Secure` attribute (lines 6), then the protocol that was used to serve the response is `HTTPS` (line 7).

We encode a query for finding the counterexample to the invariant, as described in Section III-A, with the following definition:

```
1 Inductive SecureCookiesQuery (gb: Global) (evs: list Event) (
  st: State): Prop :=
2   | Query_state : ∀ rp corr _evs cookie,
3   Reachable gb evs st →
4   evs = (EvResponse rp corr :: _evs) →
5   rp_hd_set_cookie (rp_headers rp) = Some cookie →
6   sc_secure cookie = true →
7   url_protocol (rp_url rp) ≠ ProtocolHTTPS →
8   SecureCookiesQuery gb evs st.
```

At lines 1-2 we define an inductive datatype with a single constructor, which can be created by providing all the hypotheses of the invariant (lines 3-6 of `SecureCookieInvariant`) and the negation of the conclusion (line 7, note that in the query we require the protocol to be $\neq$ `HTTPS`). We ran WebSpec on `SecureCookieQuery` and no attack could be detected up to a trace size of 50 events. Moreover, we constructed a formal proof in Coq that the invariant holds. Due to space constraints, we refer the reader to [7] for the Coq source files of the proof.

2) *Integrity of `__Host-` Cookies*: The `__Host-` cookie prefix is used to ensure that security-sensitive cookies are set as host-only, thus ensuring their integrity against same-site attackers [41]. When a cookie whose name starts with `__Host-` is set, the browser verifies that the `Domain` attribute is not present and discards the cookie otherwise. This corresponds to the following invariant.

Invariant. A `__Host-` cookie set for the domain d can be set either by d (via HTTP headers) or by scripts included by the pages on d .

We encode the invariant in our model by splitting the two cases in which a host cookie can be set: (i) via HTTP headers, and

(ii) via JavaScript. For space reasons we present only case (ii) below and refer to Section A-A1 for the full definition.

```
1 Definition HostInvariantSC (gb: Global) (evs: list Event) (st:
  State): Prop :=
2   ∀ pt sc ctx c_idx cookie cname h _evs,
3   Reachable gb evs st →
4   (* A script is setting a cookie *)
5   is_script_in_dom_path gb (st_window st) pt sc ctx →
6   evs = (EvScriptSetCookie pt (DOMPath [] DOMTopLevel) c_idx
7     cookie :: _evs) →
8   (* The cookie prefix is __Host *)
9   (sc_name cookie) = (Host cname) →
10  (* The cookie has been registered in the script
11    context *)
12  url_host (wd_location ctx) = Some h →
13  (sc_reg_domain cookie) = h.
```

For every reachable state in which a script `sc` is setting a cookie on the top-level window (lines 3-6), `ctx` is the window (browsing context) in which the script `sc` is running (line 5). If the cookie has the `__Host-` prefix (line 8), we require (line 11) the domain on which the cookie was registered to be equivalent to the domain of the `ctx` browsing context. This corresponds to stating that a script running on a page of domain d can set a host-prefix cookie only for the domain d .

Attack. When we run the query, our toolchain discovers a novel attack that breaks the invariant using domain relaxation. A script running on a page can modify at runtime the *effective* domain used for SOP checks through the `document.domain` API. Indeed, the value of `document.domain` is taken into account only for DOM access. All remaining access control policies implemented in the browser use the original domain value [38]. This is the case, for instance, for cookie jar access, XMLHttpRequests, and origin information reported when performing a `postMessage`. The mismatch between the access control policies in the DOM and the cookie jar allows a script running in an `iframe` to access the `document.cookie` property of the parent page when both pages set `document.domain` to the same value. Once the inner frame performs a set cookie of a host-prefix cookie through the parent page DOM, the browser uses the original domain value of the parent page to perform the host prefix checks, breaking the invariant.

The trace generated by WebSpec is shown in Figure 6 and detailed below. In the following, expressions of the form `DOMPath __` represent a unique path in the DOM. In particular, the first argument of `DOMPath` is the nesting level. For instance, we refer to the window loaded inside two nested iframes as `DOMPath [1,3] __`, where 1 and 3 are the indexes of the DOM elements representing the frames. The second argument is used to refer to a specific DOM object (`DOMIndex`) or to the whole document loaded in the frame (`DOMTopLevel`). An example is shown in Figure 5: the path to an image at index 3 loaded inside two nested iframes (respectively at index 2 and 1) is represented as `DOMPath [1,2] (DOMIndex 3)`, while the path of the window containing the image is `DOMPath [1,2] DOMTopLevel`.

The attack trace describes the following scenario: (steps 1-3) a page from `origin_1` is loaded in the top-level window of the browser. Note that `origin_1` is the sub-

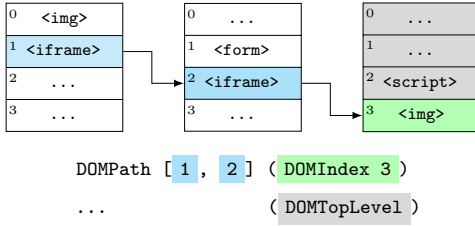


Fig. 5. DOM Path Datatype

domain named 16162 of the host 13, loaded via HTTPS; (4-6) an iframe element is loaded from `origin_4` at index 0 of the DOM in the main window (in the path `DOMPath [] (DOMIndex 0)`). Note that `origin_4` is another subdomain of the same host; (7-9) a script is loaded in the main window at index 1; (10-12) a script is loaded in the iframe at index 0 (`DOMPath [0] (DOMIndex 0)`); (13) the script in the parent window sets its `document.domain` to its parent domain 13; (14) the script in the iframe sets its `document.domain` to its parent domain 13. From now on, the two pages are effectively same origin, having performed domain relaxation to the same domain; (15) the script inside the iframe (`DOMPath [0] (DOMIndex 0)`) sets a cookie using the `document.cookie` setter of the top-level window (`DOMPath [] DOMTopLevel`). The cookie has the `__Host-` prefix and has been set by `origin_1` for `origin_2`, breaking the invariant.

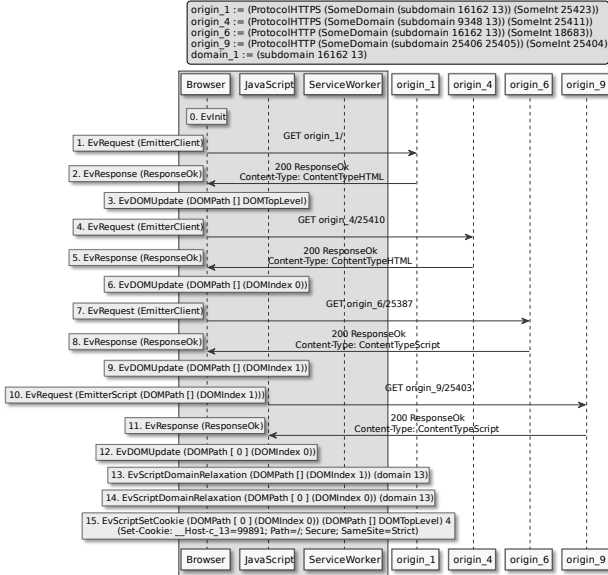


Fig. 6. Host Cookies Inconsistency

The `document.domain` setter has been deprecated in modern browsers [34] since, similarly to the counterexample produced by our toolchain it may undermine the security of other Web components. Although the current Web platform is still vulnerable to the attack we discovered, removing domain relaxation from the Web platform will eventually

make the invariant hold. WebSpec can be configured to prevent scripts from using `document.domain` by specifying `c_domain_relaxation (config gb)= false`, allowing us to verify (up to a finite size, see Section VI) that the invariant holds.

B. Content Security Policy

The *Content Security Policy* (CSP) allows Web developers to tighten the security of Web applications by controlling which resources can be loaded and executed by the browser. Originally, the CSP was designed to mitigate content injection vulnerabilities. Subsequently, it was extended to restrict browser navigation (e.g., `form-action`, `frame-ancestors`) and protect DOM XSS sinks (via `trusted-types`). A CSP policy consists of a set of directives and source expressions specifying an allow-list of actions the page is allowed to perform.

1) *Interactions with the SOP*: With the `script-src` CSP directive, developers can specify which scripts can be included in a page and thus access the DOM. This corresponds to the following property.

Invariant. *The DOM of a page protected by CSP can be read/modified only by the scripts allowed by the policy.*

We encode the invariant in our model as follows:

```
1 Definition CSPInvariant (gb: Global) (evs: list Event) (st:
  State): Prop :=
2   ∀ pt sc ctx pt_u src origin tctx tt _evs,
3   Reachable gb evs st →
4   (* A script sc is present in the page *)
5   is_script_in_dom_path gb (st_window st) pt sc ctx →
6   (* The DOM of the toplevel window has been modified
     by sc *)
7   evs = (EvScriptUpdateHTML pt (DOMPath [] pt_u) tctx :: _evs )
8   →
9   (* The toplevel window is protected by CSP *)
10  rp_hd_csp (dc_headers (wd_document (st_window st))) = Some
11  { | csp_script_src := Some src; csp_trusted_types := tt | }
12  →
13  (* The script sc is allowed by the CSP *)
14  origin_of_url (wd_location (st_window st)) = Some origin
15  →
16  csp_src_match src origin (script_src sc).
```

Where the `csp_src_match` predicate holds when the `src` source expression matches the URL `script_src sc` in a page loaded from origin `origin`.

Attack. By running the query, our toolchain produces a counterexample that corresponds to the CSP violation discovered by the authors of [39]. The complete trace is shown in Figure 7: (steps 0-2) a page with `Content-Security-Policy: script-src 'none'` is loaded. The `none` value specifies that no script is allowed to be included in this page; (3-5) the page contains a same-origin (`origin_2`) iframe with `script-src origin_3` as CSP, allowing the page loaded in the iframe to (6-8) include scripts from `origin_3`; (9) the script running in the iframe (that was loaded from `origin_3`) can access the DOM of the parent page and modify it, which is allowed by SOP since the two pages come from the same origin. This is particularly dangerous in case the framed page is either compromised or malicious since any attacker-provided script could access the

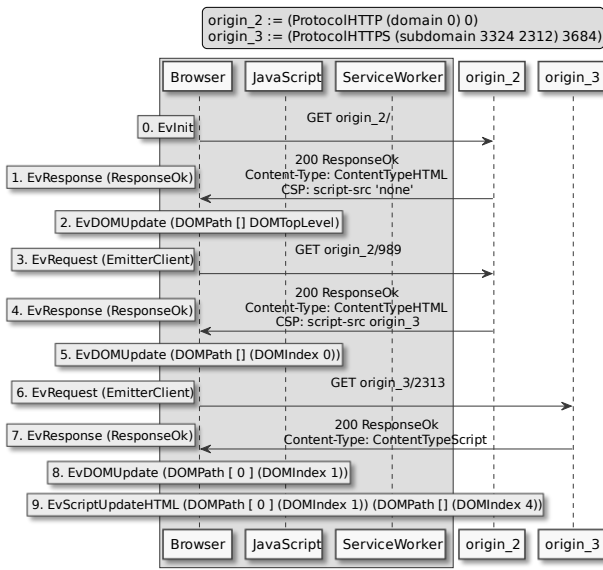


Fig. 7. CSP Inconsistency

content of the parent page. Similar issues can arise when the framed page is protected by CSP while the parent is not or when the two pages have different origins (but the same site) and domain relaxation is performed [39].

Preventing similar CSP violations could be achieved by having the same CSP policy enforced on all same-origin pages in a site and by disabling domain relaxation (e.g., by removing support for the `document.domain` setter). Using an origin-wide CSP policy can be done manually or via the upcoming Origin Policy [22] mechanism when it will be supported by major browsers. We can configure our model to apply the same CSP policy to all same-origin pages with the `c_origin_wide_csp (config gb) = true` configuration option and verify that the invariant holds. The Coq proof of the correctness of this solution is available at [7].

2) *Integrity of server-provided policies:* A service worker [49] is an event-driven worker that acts as a client-side proxy between Web applications and the network. Service workers are intended to enable Web applications to be used even without a network connection. They can intercept and modify network requests towards the origin against which they are registered and all requests triggered by the pages hosted on that origin. Using the Cache API, service workers can be used to store HTTP responses and then serve them even when the network is unreachable.

Given the position of service workers in the processing of requests and responses, we must ensure that if a response obtained from the network contains a security policy, then such policy cannot be tampered with by the network stack and is correctly enforced by the browser. This corresponds to the following Web invariant.

Invariant. *If a response from the server contains a security policy, then the browser enforces that specific policy.*

We encode the invariant in our model as follows:

```
1 Definition SWInvariant (gb: Global) (evs: list Event) (st:
  State): Prop :=
2   ∀ corr rq_idx rp_idx rp em,
3   Reachable gb evs st →
4   (* Get the server response *)
5   is_server_response gb rq_idx rp →
6   (* Get the response that was rendered *)
7   in_history (st_fetch_engine st) corr (em, rq_idx, rp_idx) →
8   (* The CSP of the rendered response is equal to the
       server one *)
9   rp_hd_csp (rp_headers rp) =
10  rp_hd_csp (rp_headers ((responses gb), [rp_idx])).
```

For every response `rp` that would be generated by the server for a specific request index `rq_idx`, the response that has been rendered by the browser is present in the `ft_history` field of the `FetchEngine`. In particular, the history stores the mapping between requests and responses (`rq_idx, rp_idx` at line 7) for every response that is rendered by the browser. The invariant requires that the CSP of the response that is present in the history must be the same as the one that is generated by the server.

Attack. Running the query on WebSpec reveals that it is indeed possible for a service worker to break the invariant by responding to a request with a synthetic response (i.e., created with the `Response` constructor). In particular, when the server-generated response contains a security policy, a service worker could discard the network response and respond to the request with a new possibly unrelated response. This corresponds to an *inattentive* service worker which, due to a programming error, might remove or relax the security policies that are part of the responses the service worker is handling. We can specify that service workers are not allowed to generate synthetic responses using the `c_worker_allow_synthetic_responses (config gb) = false` configuration option. This configuration allow us to model the *cache-first* or *offline-first* pattern, the most popular¹ programming pattern that is used to serve content using service workers. An *offline-first* service worker intercepts all network requests: if a resource is found in the cache, then it is returned to the user before trying to download it; otherwise, if a resource is not found in the cache, the resource is fetched from the network and added to the cache.

When we run the query again in this configuration, WebSpec produces a counterexample (shown in Figure 8): the invariant is broken once again when a service worker returns a synthetic response that has been added to the cache. Here, however, the synthetic response has been added to the cache by a script running on a page that is same-origin with the service worker. In particular, at step 6, a script running on `origin_2` creates a new response object that does not contain any security header and adds it to the cache. When the browser fetches `origin_4/`, the service worker matches the response that was previously cached by the script and returns it instead of downloading it. So the response rendered by the browser has a different CSP than the original response returned by the

¹https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Offline_Service_workers

server, breaking the invariant. This is a special case of the

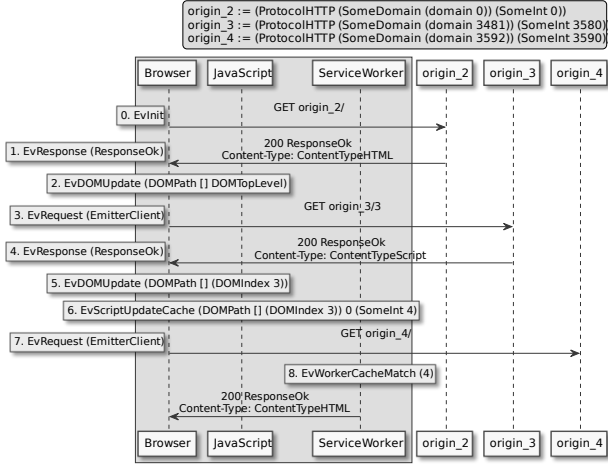


Fig. 8. Service Workers Cache Inconsistency

attack described by Squarcina et al. [40], where an attacker tampers with cached responses to strip or weaken the CSP served to the user. As the authors pointed out, this issue can be prevented by making the Cache API inaccessible to scripts running in the page context. We can verify that the invariant holds by restricting the Cache API to workers only, using the `c_script_update_cache (config gb)= false` configuration option. The security proof of this fix is available online [7].

3) *Access control on Trusted Types DOM sinks*: Trusted Types is an experimental API that allows applications to restrict DOM XSS sinks, accepting only non-spoofable typed values in place of strings. These types can be created based on application-defined policies, allowing developers to specify rules to protect injection sinks.

Trusted Types are controlled by two CSP directives: `require-trusted-types-for 'script'`, which enables the enforcement of Trusted Types, i.e., instructs the browser to only accept Trusted Types for all DOM XSS injection sinks, and `trusted-types`, optionally followed by the name of one or more policies, which specifies the policies (part of the application Javascript) that are allowed to create Trusted Types objects. When no name is specified or when the special value `'none'` is used, no policy, and thus no Trusted Type, can be created. In the latter case, when enforcement is enabled, DOM XSS sinks are effectively disabled. This corresponds to the following invariant.

Invariant. *If a page has both `trusted-types`; and `require-trusted-types-for 'script'`; directives in the CSP then no script in the page can modify the DOM using a Trusted Types sink.*

We encode the invariant in our model as follows:

```
1 Definition TTInvariant (gb: Global) (evs: list Event) (st:
  State): Prop :=
2   ∀ pt target_pt target_ctx ssrc ttypes,
3     Reachable gb evs st →
```

```
4   (* The target context has Trusted-Types enabled *)
5   url_protocol (wd_location target_ctx) = ProtocolHTTPS →
6   rp_hd_csp (dc_headers (wd_document target_ctx)) = Some
7     { csp_script_src := ssrc; csp_trusted_types := Some
        ttypes } →
8   tt_policy ttypes = Some None →
9   tt_require_for_script ttypes = true →
10  (* No script can update the dom using innerHTML *)
11  not (In (EvScriptUpdateHTML pt target_pt target_ctx) evs).
```

Here we assert that there cannot be an html update event (using a DOM XSS sink, e.g., `innerHTML`) for the window `target_ctx`, if the aforementioned directives are used to define the policy for `target_ctx`.

Attack. An earlier version of the Trusted Types draft [30, Editor’s Draft, 3 February 2021] restricted Trusted Types to Secure Contexts only. This was part of an effort of browser vendors to restrict all new APIs to secure contexts to help advance the Web platform to default to the HTTPS protocol. The restriction, however, enabled attackers to bypass Trusted Types by framing the protected page from a non-secure context [5]. This silently disabled the DOM XSS protection despite the fact that the document was downloaded using a secure connection. When we enable the secure context restriction in our model, WebSpec is able to rediscover the bypass.

We can disable the secure context restriction with the `c_restrict_tt_to_secure_contexts (config gb)= false` configuration option. However, when we run the solver again with this configuration, our toolchain is still able to find a counterexample for which the invariant does not hold. The trace is shown in Figure 9: (steps 1-3) a page protected with Trusted Types is loaded from `origin_1`. In particular, no policy is allowed, so no Trusted Type can be created; (4-6) the page contains a same-origin iframe which specifies a Trusted Types policy (`trusted-types 25809`), allowing the scripts loaded in this iframe to create Trusted Types using a policy named `25809`; (7-9) a script that is loaded in the iframe modifies (10) the DOM of the parent frame using a Trusted Types sink. This is possible because the inner frame is able to create Trusted Types that are accepted by all DOM XSS sinks and because, being same origin, the inner frame can access the DOM of the parent. A similar attack on related domains is possible if the parent page performs domain relaxation, as the value of `document.domain` is used for DOM access control.

The Trusted Types draft [30, §5.1] includes a brief discussion of a similar attack in which cross-document import of nodes would bypass the enforcement of the policy. However, the current specification does not provide any solution and suggests that other mechanisms like *Origin Policy* [22] might be used to ensure that the same policy is deployed across the whole origin. Instead, we propose a different solution based on *non-transferable* Trusted Types, and prove the correctness of our approach within our model in Section V.

4) *Safe policy inheritance*: The Content Security Policy specification [45, §7.8] mandates that every document that is loaded from a local scheme must inherit a copy of the policies of the source browsing context, that is, the browsing

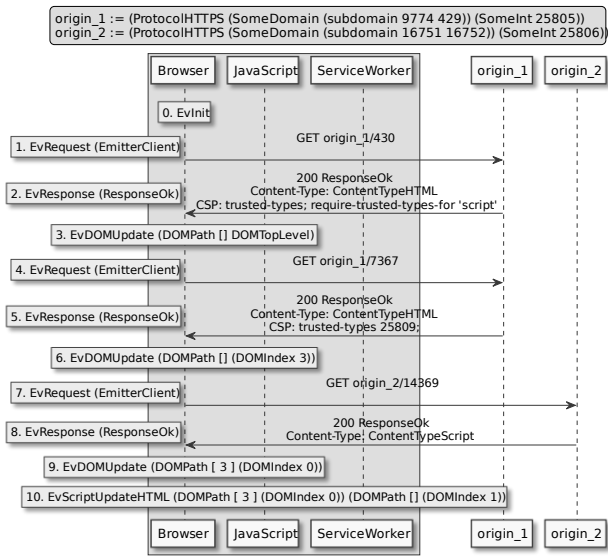


Fig. 9. Trusted-types bypass with same-origin iframes

context that was responsible for starting the navigation. This corresponds to the following invariant.

Invariant. *Documents loaded from a local scheme inherit the policy of the source browsing context.*

We encode the invariant in our model as follows:

```
1 Definition LSInvariant (gb: Global) (evs: list Event) (st:
  State) : Prop :=
2   ∀ evs pt _evs frm fhtml fwd ctx lv pt_idx init_idx,
3   let get_csp wd :=
4     rp_hd_csp (dc_headers (wd_document wd)) in
5   Reachable gb evs st →
6   (* A document has just been loaded in a frame *)
7   evs = (EvDOMUpdate pt :: _evs) →
8   is_frame_in_dom_path gb (st_window st) pt frm fhtml fwd
9   ctx →
10  is_local_scheme (wd_location fwd) →
11  (* get navigation initiator *)
12  pt = DOMPath lv (DOMIndex pt_idx) →
13  is_wd_initiator_of_idx ctx pt_idx (Some init_idx) →
14  (* The csp is equal to the req. initiator *)
15  get_csp fwd = get_csp (windows gb.[init_idx]).
```

When a frame has just loaded a document from a local scheme (lines 7-9), we require that the CSP of the navigation initiator (i.e., the source browsing context) is equal to the policy of the document loaded in the frame window (line 14).

The goal of this Web invariant is to ensure that a page cannot bypass its policy by navigating to content that is completely under its control. One such bypasses [2] was caused by the behavior defined for the inheritance of policies in a previous version of the CSP specification [45, 15 October 2018]: documents loaded from local schemes would inherit the policies of the embedding document or the opener browsing context.

Recently, the concept of *policy container* was added to the HTML specification [52, §7.9]. A policy container is a collection of policies to be applied to a specific document and its purpose is to simplify the initialization and inheritance of policies. The introduction of the policy container in the

specification allowed for clarifying the inheritance behavior for local schemes, which might differ depending on the specific scheme or URL that is used. The policy container explainer [3] stipulates the following behavior:

about:srcdoc An `iframe` element with the `srcdoc` attribute inherits the policies from the embedding document, i.e., the parent frame. Note that `srcdoc` iframes are in the same origin of the embedding document but their location URL is `about:srcdoc`.

about:, data: A document loaded from the `data:` or `about:` schemes inherits the policies of the navigator initiator (as mandated by the CSP specification).

blob: A document loaded from a `blob:` URL inherits the policies from the document that creates the URL, i.e., the document that calls the `URL.createObjectURL` function.

Note that in the current version of the HTML specification [52, §7.11.1] the inheritance behavior for `blob:` URLs matches the one for `about:` and `data:`, thus following the CSP specification. We contacted the editors of the HTML specification [4] asking for a clarification on the correct behavior for `blob:` URL and they confirmed that, because of the wrong ordering of a clause in the policy container construction for blobs, the initiator policy container was always replacing the creator policy container. The correct inheritance rule is to inherit the policy container of the creator of the URL [1], thus introducing an inconsistency between the CSP specification and the HTML specification (as `blob:` is a local scheme that is handled differently from the others).

Attack. When we configure our model to reflect a past state of the Web platform in which policies were inherited from the embedding frame and not from the navigation initiator (`c_csp_inherit_local_from_initiator (config gb) = false`), our toolchain is able to rediscover the attack trace that allows an attacker to strip the CSP policies by navigating a frame to a local scheme URL. The trace is shown in Figure 10: (steps 1-6) a document with no Content Security policy loads an `iframe` with a restrictive CSP; (7-9) the `iframe` contains a script which navigates (10) the frame itself (e.g., using the `window.location` setter) to a local scheme URL; (11-13) the `iframe` renders the content of the local scheme URL and inherits the CSP from the embedding document, which does not contain any policy. The resulting document has no CSP, effectively removing the policy that was previously defined for the `iframe`.

We can configure our model to reflect the current state of the Web platform by inheriting the policies from the navigation initiator for all local schemes:

```
c_csp_inherit_local_from_initiator (config gb) = true ∧
c_csp_inherit_blob_from_creator (config gb) = false.
```

We can verify (up to a finite size, see Section VI) that with this configuration the invariant holds. However, when we configure our model to reflect the planned modification of inheriting the policies of the URL creator when rendering a

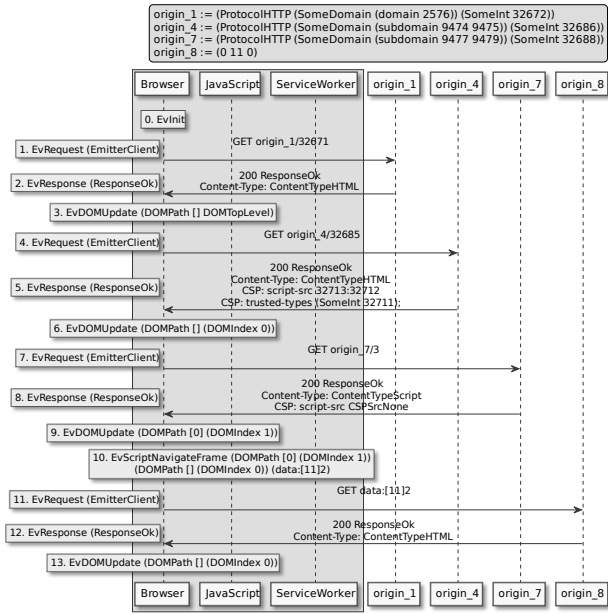


Fig. 10. CSP bypass due to inheritance from the embedder document

blob: URL (c_csp_inherit_blob_from_creator (config gb)=true), Z3 is able to find a new counterexample. The trace is similar to the one depicted in Figure 10 with an additional script loaded in the top-level window executing an `EvScriptCreateBlobUrl` event: (i) a page in `origin_1` with no CSP loads a same-origin iframe with a restrictive policy; (ii) a script running on the embedding document creates a new blob URL (`EvScriptCreateBlobUrl`); (iii) a script running on the inner frame navigates the frame itself (i.e., setting `window.location`) to the previously created URL. The frame loads the content of the blob and inherits the CSP from the embedding document, which does not have any policy. Similarly to the previous attack trace, the policy that was defined for the iframe has been removed by navigating to local-scheme content.

Hence, the planned modification on CSP inheritance in the HTML standard would introduce an inconsistency with the CSP specification. We responsibly reported the issue to the working group of the HTML standard [4], who initially deemed the security implications of the attack as low. However, at the time of writing (December 2021), no final decision has been taken and the current browser behavior remains unchanged, i.e., our invariant still holds.

V. VERIFICATION OF WEB SECURITY PROPERTIES

In this section, we show how WebSpec can be used to formally verify the security of a fix to the attack against Trusted Types presented in Section IV-B3. Due to lack of space, we do not discuss in the paper the other 3 security proofs that we have developed, yet their source code is available online at [7].

According to the current draft of the Trusted Types specification, a Trusted Type object created by a page can be assigned to DOM XSS sinks belonging to different pages. This allows for bypassing the protection if a restricted document colludes with an unrestricted one. This can happen, e.g., in case of same-origin iframes (see Section IV-B3). The specification acknowledges the issue and suggests the usage of the Origin Policy [22] to address the problem, which unfortunately is not currently supported by any browser.

For this reason, we propose an alternative solution we label *non-transferable* Trusted Types, which consists in labeling each Trusted Type with the JavaScript realm (window or worker) that created it and ensuring that a type can only be assigned to DOM XSS sinks from the same realm. Therefore, our fix effectively prevents cross-document usage of Trusted Types. We implemented this behavior in WebSpec, which can be activated by setting the configuration option `c_tt_strict_realm_check` to `true`. The following theorem states the validity of the invariant `TTInvariant` (cf. Section IV-B3) when our fix is enabled:

```
1 Theorem strict_realm_check_implies_invariant :
2   ∀ gb evs st,
3     c_tt_strict_realm_check (config gb) = true →
4     c_restrict_tt_to_secure_contexts (config gb) = false →
5     TTInvariant gb evs st.
```

We recall that, according to the invariant, if a page is shipped with a CSP containing the directives `trusted-types` and `require-trusted-types-for 'script'`, then the list of events `evs` cannot contain a `EvScriptUpdateHTML` event that updates the contents of the page.

We can prove the theorem by induction on the `Reachable` relation where all the cases except `EvScriptUpdateHTML` are trivial. In this latter case we show that, by enabling strict realm checking, it is impossible to generate the correct Trusted Type for the update, since (i) the `trusted-types` directive disallows the creation of Trusted Types for the realm in which the directive is used; and (ii) the only Trusted Types that are accepted by a page with `require-trusted-types-for 'script'` are only those labelled with the realm of the page. This suffices to prove the correctness of the proposed solution within our model.

VI. EVALUATION

In our experimental evaluation, we used WebSpec to automatically discover the attacks reported in Section IV. Additionally, when we implemented a fix to an attack, we ran WebSpec again to confirm that the issue had been addressed. Since the μZ solver may not terminate (see Section III), we use the length of the previously discovered attack trace plus one as the maximal search size, thus verifying that the previous attack is not reachable anymore.

In this section, we report the time required by WebSpec to find the attacks and describe various optimization techniques that allowed us to drastically improve the performance of our approach. All our experiments have been conducted on a virtual machine with 32 VCPUs (2GHz AMD EPYC) and 128GB of RAM.

TABLE II
TRACE SIZE AND SOLVING TIME FOR EACH ATTACK

#	Query	Trace Size # Events	Solving Time			
			Frames Enabled		Frames Disabled	
			Baseline	w/ Lemmas	Baseline	w/ Lemmas
1	Integrity of <code>__Host-</code> cookies	15	58d 1h 30m	23m *	×	×
2	Confidentiality of <code>HttpOnly</code> cookies	7	13h 35m	8m	1h 46m	1m
3	Interaction between SOP and CSP	10	42d 3h	46m *	×	×
4	Integrity of server-provided responses	8	40h 35m	18m	15h 46m	3m
5	Access control on Trusted Types sinks	9	17h 50m	–	×	×
6	Access control on Trusted Types sinks (no sec. ctx)	10	9d 20h 18m	–	×	×
7	Safe policy inheritance (inherit from parent)	13	52d 10h	1h 48m	×	×
8	Safe policy inheritance (inherit from creator)	17	–	6h 5m	×	×
9	Authenticity of request Initiator	5	1h 49m	–	14m	–
10	Authorization of non-simple requests (i)	5	35m	–	5m	–
11	Authorization of non-simple requests (ii)	10	–	48m	35d 10h 51m	7m

×: N/A; – (Baseline): No solution could be found within 60 days; – (w/Lemmas): None of our user-defined lemmas could be applied;
*: a lemma has been automatically extracted from the attack trace of a previous run of the solver.

The baseline performance is displayed on the third column of Table II. We can observe a clear correlation between the size of the attack trace and the time required to find an attack, which is caused by the unrolling technique employed by the BMC engine of the μZ solver [28] used in WebSpec. In particular, time increases exponentially with respect to the size of the attack trace, leading to running times of several days or weeks for traces with 10 or more events.

To tackle these performance issues, we have implemented various optimizations that consist in (i) defining additional rules (or lemmas) representing common configurations (e.g., loading of a frame containing a script) that can be used by the SMT solver instead letting it figure out on its own the right list of events leading to these configurations, and (ii) simplifying the model at compile time (e.g., by disabling frames) so that the resulting SMT-formula is easier to solve.

a) User-defined lemmas: The key idea underlying this optimization is to enable users to define additional lemmas that guide the solver into constructing interesting browser states that can be used as a starting point to discover attacks.

Consider the following example:

```
1 Lemma script_state_is_reachable : ∀ gb,
2   script_state_constraints gb →
3   Reachable gb (script_state_events gb) script_state.
4 Proof. [...]
```

Here `script_state` is a Coq definition of a concrete browser state where a script is loaded in the page rendered in the top-level window. The lemma says that the state is reachable by applying the list of events `script_state_events` assuming that `script_state_constraints` is satisfied. Once it can be proved that the state is `Reachable`, the lemma can be compiled together with the query into CHC logic.

Since the BMC engine solves queries by iterative unrolling, it prioritizes the rules that result in the smallest amount of unrolling steps. Lemmas exploit this property by providing a one-step solution for the generation of states that would require multiple steps if the solver had to build them from scratch.

The results of this optimization are highlighted in Table II, where we can see that the usage of lemmas always reduces

the runtime to less than a day. It may however happen that the solver is not able to apply any of the user-defined lemmas, as it is the case for queries 5-6 (marked with –). In such cases no performance improvement can be obtained. For the queries marked with *, we automatically extracted a lemma from an attack trace discovered by WebSpec and confirmed that it can be used by successive runs of the solver. The extraction of lemmas from traces could allow us to create a library of reachable browser configurations that can be used to improve the resolution time of new queries. We leave the creation of this library and the definition of a methodology to generate generic lemmas as future work.

b) Configurable inlining of auxiliary relations: Our model relies on a `Reachable` relation that models state transitions, a `ScriptState` relation that models scripts knowledge, and multiple auxiliary relations that are used within `Reachable` to, e.g., recursively update the DOM.

The presence of multiple relations prevents us to directly use the best performing version of the BMC engine, the linear solver, because it requires the model to be encoded as linear Horn clauses, i.e., clauses containing at most one recursive term. In order to satisfy this requirement, every auxiliary relation needs to be inlined within the main `Reachable` relation. To this end, WebSpec automatically unrolls all the applications of recursive relations that are marked for inlining. For each relation we specify the depth of the unrolling. For instance, the declaration

```
Inline Relation is_script_in_dom_path With Depth 3.
```

says that the relation `is_script_in_dom_path`, which searches a script inside the DOM, must be unrolled up to recursion level 3. Depth 0 disables all recursive calls and expands the relation to the base case only. For instance, support for nested frames can be easily deactivated by specifying 0 as depth for all the relations handling the DOM tree.

The recursion depth affects the solving time of μZ since multiple applications of the relation need to be considered. Disabling nested frames for the queries which do not require them simplifies the compiled model and allows for faster solv-

ing. When frames are required, we set the `Depth` parameter so that a single level of nesting is allowed. Although our model can handle an arbitrary number of nested frames, a single level suffices to discover the minimum-size trace for all queries.

The effects of this optimization are shown in Table II: in particular we can see that disabling framing for the queries that do not require nested DOM trees considerably lowers the solving time.

c) Fixed size arrays: Our model makes use of functional arrays [31], [37] in several places: in the bodies of HTML and DOM objects, for example, or in some implementation details like the flattening of the window/frame tree. However, these functional arrays are known for significantly increasing the complexity of queries [44], [43]. Therefore, in order to ease the resolution, our compiler provides an optimization which turns functional arrays into arrays of a fixed size chosen at compile time. Because choosing a small size can make a query unsolvable, we launch in parallel several instances of the same query with different sizes and keep the first one to succeed. Surprisingly, a size of 5 is enough for all the query except those for *Safe policy inheritance* which requires a size of 7.

VII. RELATED WORK

In the past years, researchers applied formal methods to automatically find bugs and to formally verify the security of existing Web mechanisms and protocols. In the following we discuss the most relevant works in the field and critically compare them with our proposal.

a) Models of the Browser: In his PhD thesis, Bohannon [15] proposed Featherweight Firefox, a model of a Web browser written in Coq for the verification of security properties concerning JavaScript execution. For this reason, the model supports many JavaScript features, such as DOM manipulation, XHR requests, event listeners and code evaluation via the `eval` function. On the other hand, the set of modeled Web components is rather narrow: the model supports windows, cookies and HTML, whose support is however limited to `<script>` and `<div>` tags.

Bugliesi et al. [16] extended Featherweight Firefox to formalize the security guarantees conveyed by the usage of the cookie flags `HttpOnly` and `Secure` with respect to network attackers and Web attackers able to exploit cross-site scripting (XSS) vulnerabilities. In [17] the authors use Featherweight Firefox as a starting point to develop a pen-and-paper model of a security-enhanced browser which enforces a Web session integrity property that captures attacks like cross-site request forgery (CSRF) and theft of credentials via XSS.

In contrast to WebSpec, Bohannon’s model and later extensions were developed with machine-checked proofs in mind and have not been used to automatically detect vulnerabilities. They also lack support for most of the Web features considered in our invariants, e.g., CORS, CSP, service workers.

b) Models of the Web: In their seminal work, Akhawe et al. [8] developed in Alloy the first formal model of the Web ecosystem. The authors encoded in the model a set of security goals, which include fundamental properties of the

Web platform that are assumed to hold, and a notion of session integrity capturing CSRF attacks. The validity of these goals has been checked with the Alloy Analyzer and their violations pointed out the existence of novel and previously known attacks. Compared to our model, the one of Akhawe et al. cannot be used to prove security properties since the Alloy Analyzer uses SAT-based bounded model checking, but just to disprove them. Additionally, having being developed in 2010, it lacks many features of the modern Web (e.g., CSP and service workers) that are a fundamental part of our model.

Bansal et al. [9] developed WebSpi, a generic library that defines the basic components of the Web infrastructure (browsers, HTTP servers) and enables developers to encode specific Web applications / protocols and security properties which can be automatically verified using ProVerif [14]. The browser model of WebSpi is rather primitive and includes only a subset of the features supported in WebSpec. This is in line with the intended usage of WebSpi, that is, the verification of Web protocols, for which it suffices to model only the features used by the protocol under analysis. In this work, instead, we target inconsistencies between Web features themselves, without focusing to a specific Web protocol or application, for which we need a much more comprehensive browser model, which is hardly amenable to automated verification. Instead, we provide support for machine-checked proofs in Coq.

The most comprehensive and maintained model of the Web to date is the *Web Infrastructure Model* (WIM), a pen-and-paper model which has been used to assess the security of a variety of Web protocols, including OAuth 2.0 [25], OpenID Connect [26], and the Financial-Grade APIs [24]. The browser model of WebSpec supports all the features of WIM browsers, with the exception of (i) HSTS, since in our model we abstract away from the network, and (ii) HTTP basic authentication, because it is an application-specific authentication mechanism enforced on the server side. On the other hand, WebSpec supports several client-side mechanisms and security policies, like domain relaxation, CSP and CORS, that are not part of WIM. Additionally, being a pen-and-paper model, WIM can neither be used to automatically discover security vulnerabilities, nor to develop computer-assisted proofs, which are central features of our framework.

VIII. CONCLUSION

In this paper we presented WebSpec, the first formal framework for the security analysis of Web features that supports the automated detection of logical flaws and allows the development of machine-checked security proofs. We showcased the effectiveness of WebSpec by discovering novel attacks and inconsistencies in the modern Web and by highlighting how previously reported attacks could have been spotted by our framework. Additionally, we discussed how WebSpec can be used to carry out machine-checked security proofs for vulnerability fixes.

As a future work, besides expanding the model to encompass more Web platform components, we are planning to develop a generic library of relevant browser configurations

that can be used as starting points for the verification of Web invariants. In this way, we can guide the resolution strategy of the Z3 solver to verify the property on interesting test cases and, as a result, substantially improving the performance of our approach.

REFERENCES

- [1] “Fix policy container construction for blobs,” <https://github.com/whatwg/html/pull/6895>.
- [2] “Issue 894228: CSP bypass with blob URL,” <https://bugs.chromium.org/p/chromium/issues/detail?id=894228>.
- [3] “Policy container explained,” <https://github.com/antosart/policy-container-explained>.
- [4] “Should blob: inherit CSP in addition to origin?” <https://github.com/whatwg/html/issues/2593#issuecomment-885083373>.
- [5] “Trusted-types: Restrict to secure contexts,” <https://github.com/w3c/webappsec-trusted-types/issues/259#issuecomment-630863753>.
- [6] “The web platform tests project,” <https://web-platform-tests.org/>.
- [7] “WebSpec: Coq proofs and source files,” <https://github.com/SecPriv/webspec>.
- [8] D. Akhawe, A. Barth, P. E. Lam, J. C. Mitchell, and D. Song, “Towards a Formal Foundation of Web Security,” in *Proceedings of the 23rd IEEE Computer Security Foundations Symposium, CSF 2010*, 2010, pp. 290–304.
- [9] C. Bansal, K. Bhargavan, A. Delignat-Lavaud, and S. Maffei, “Discovering Concrete Attacks on Website Authorization by Formal Analysis,” *Journal of Computer Security*, vol. 22, no. 4, pp. 601–657, 2014.
- [10] H. Barbosa, A. Reynolds, D. Larraz, and C. Tinelli, “Extending enumerative function synthesis via smt-driven classification,” in *2019 Formal Methods in Computer Aided Design, FMCAD 2019, San Jose, CA, USA, October 22-25, 2019*, 2019, pp. 212–220.
- [11] H. P. Barendregt, *Lambda Calculi with Types*, 1992, p. 117–309.
- [12] A. Barth, “HTTP State Management Mechanism,” Internet Requests for Comments, Internet Engineering Task Force, RFC 6265, 4 2011. [Online]. Available: <https://tools.ietf.org/html/rfc6265>
- [13] A. Barth, C. Jackson, and J. C. Mitchell, “Robust Defenses for Cross-Site Request Forgery,” in *Proceedings of the 15th ACM Conference on Computer and Communications Security, CCS 2008*, 2008, pp. 75–88.
- [14] B. Blanchet, “An Efficient Cryptographic Protocol Verifier Based on Prolog Rules,” in *Proceedings of the 14th IEEE Computer Security Foundations Workshop, CSFW 2001*, 2001, pp. 82–96.
- [15] A. Bohannon, “Foundations of Webscript Security,” Ph.D. dissertation, University of Pennsylvania, 2012.
- [16] M. Bugliesi, S. Calzavara, R. Focardi, and W. Khan, “CookiExt: Patching the browser against session hijacking attacks,” *Journal of Computer Security*, vol. 23, no. 4, pp. 509–537, 2015.
- [17] M. Bugliesi, S. Calzavara, R. Focardi, W. Khan, and M. Tempesta, “Provably sound browser-based enforcement of web session integrity,” in *Proceedings of the 27th IEEE Computer Security Foundations Symposium, CSF 2014*, 2014, pp. 366–380.
- [18] O. Community, “Httponly cookies,” <https://owasp.org/www-community/HttpOnly>.
- [19] L. Czajka, “Practical proof search for coq by type inhabitation,” in *Automated Reasoning - 10th International Joint Conference, IJCAR 2020, Paris, France, July 1-4, 2020, Proceedings, Part II*, 2020, pp. 28–57.
- [20] Ł. Czajka and C. Kaliszyk, “Hammer for coq: Automation for dependent type theory,” *Journal of Automated Reasoning*, vol. 61, no. 1–4, pp. 423–453, 2018.
- [21] O. Danvy, K. Malmkjær, and J. Palsberg, “Eta-expansion does the trick,” *ACM Trans. Program. Lang. Syst.*, vol. 18, pp. 730–751, 1996.
- [22] D. Denicola and M. West, “Origin Policy,” <https://wicg.github.io/origin-policy/>.
- [23] A. Dudenhefner and J. Rehof, “A simpler undecidability proof for system F inhabitation,” in *24th International Conference on Types for Proofs and Programs, TYPES 2018, June 18-21, 2018, Braga, Portugal*, 2018, pp. 2:1–2:11.
- [24] D. Fett, P. Hosseini, and R. Küsters, “An Extensive Formal Security Analysis of the OpenID Financial-Grade API,” in *Proceedings of the 40th Symposium on Security and Privacy, S&P 2019*. IEEE, 2019, pp. 453–471.
- [25] D. Fett, R. Küsters, and G. Schmitz, “A Comprehensive Formal Security Analysis of OAuth 2.0,” in *Proceedings of the 23rd ACM Conference on Computer and Communications Security, CCS 2016*, 2016, pp. 1204–1215.
- [26] —, “The Web SSO Standard OpenID Connect: In-depth Formal Security Analysis and Security Guidelines,” in *Proceedings of the 30th IEEE Computer Security Foundations Symposium, CSF 2017*, 2017, pp. 189–202.
- [27] K. Hoder and N. Bjørner, “Generalized property directed reachability,” in *Theory and Applications of Satisfiability Testing - SAT 2012 - 15th International Conference, Trento, Italy, June 17-20, 2012. Proceedings*. Springer, 2012, pp. 157–171.
- [28] K. Hoder, N. Bjørner, and L. M. de Moura, “μZ- an efficient engine for fixed points with constraints,” in *Computer Aided Verification - 23rd International Conference, CAV 2011, Snowbird, UT, USA, July 14-20, 2011. Proceedings*. Springer, 2011, pp. 457–462.
- [29] T. Johnsson, “Lambda lifting: Transforming programs to recursive equations,” in *Functional Programming Languages and Computer Architecture, FPCA 1985, Nancy, France, September 16-19, 1985, Proceedings*. Springer, 1985, pp. 190–203.
- [30] K. Kotowicz and M. West, “Trusted Types,” <https://w3c.github.io/webappsec-trusted-types/dist/spec/>.
- [31] J. McCarthy, “Towards a mathematical science of computation,” in *Information Processing, Proceedings of the 2nd IFIP Congress 1962, Munich, Germany, August 27 - September 1, 1962*, 1962, pp. 21–28.
- [32] M. T. Morazán and U. P. Schultz, “Optimal lambda lifting in quadratic time,” in *Implementation and Application of Functional Languages, 19th International Workshop, IFL 2007, Freiburg, Germany, September 27-29, 2007. Revised Selected Papers*, O. Chitil, Z. Horváth, and V. Zsók, Eds., 2007, pp. 37–56.
- [33] Mozilla Developers Network, “Cross-Origin Resource Sharing,” https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS#Examples_of_access_control_scenarios.
- [34] —, “Document.domain,” https://developer.mozilla.org/en-US/docs/Web/API/Document/domain#browser_compatibility.
- [35] —, “HTTP response status codes: 302 Found,” <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/302>.
- [36] A. Reynolds, H. Barbosa, A. Nötzli, C. W. Barrett, and C. Tinelli, “cvc4sy: Smart and fast term enumeration for syntax-guided synthesis,” in *Computer Aided Verification - 31st International Conference, CAV 2019, New York City, NY, USA, July 15-18, 2019, Proceedings, Part II*, 2019, pp. 74–83.
- [37] J. C. Reynolds, “Reasoning about arrays,” *Commun. ACM*, vol. 22, no. 5, pp. 290–299, 1979.
- [38] K. Singh, A. Moshchuk, H. J. Wang, and W. Lee, “On the Incoherencies in Web Browser Access Control Policies,” in *Proceedings of the 31st IEEE Symposium on Security and Privacy, S&P 2010*, 2010, pp. 463–478.
- [39] D. F. Somé, N. Bielova, and T. Rezk, “On the Content Security Policy Violations due to the Same-Origin Policy,” in *Proceedings of the 26th International Conference on World Wide Web, WWW 2017*, 2017, pp. 877–886.
- [40] M. Squarcina, S. Calzavara, and M. Maffei, “The Remote on the Local: Exacerbating Web Attacks Via Service Workers Caches,” in *15th IEEE Workshop on Offensive Technologies, WOOT 21*, 2021.
- [41] M. Squarcina, M. Tempesta, L. Veronese, S. Calzavara, and M. Maffei, “Can I Take Your Subdomain? Exploring Same-Site Attacks in the Modern Web,” in *30th USENIX Security Symposium*, 2021.
- [42] M. Steffens and B. Stock, “PMForce: Systematically Analyzing PostMessage Handlers at Scale,” in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS 2020*, 2020.
- [43] A. Stump, C. W. Barrett, D. L. Dill, and J. R. Levitt, “A decision procedure for an extensional theory of arrays,” in *16th Annual IEEE Symposium on Logic in Computer Science, Boston, Massachusetts, USA, June 16-19, 2001, Proceedings*, 2001, pp. 29–37.
- [44] N. Suzuki and D. Jefferson, “Verification decidability of presburger array programs,” *J. ACM*, vol. 27, no. 1, pp. 191–205, 1980.
- [45] W3C, “Content Security Policy Level 3,” <https://w3c.github.io/webappsec-csp/>.
- [46] —, “File API,” <https://www.w3.org/TR/FileAPI/>.
- [47] —, “Cross-Origin Resource Sharing,” <https://www.w3.org/TR/2009/WD-cors-20090317/#cross-origin-request-with-preflight0>, 2009.

- [48] —, “Cross-Origin Resource Sharing,” <https://www.w3.org/TR/cors/#generic-cross-origin-request-algorithms>, 2014.
- [49] —, “Service Workers 1,” <https://www.w3.org/TR/service-workers/>, 2019.
- [50] M. West, “Cookie Prefixes,” <https://tools.ietf.org/html/draft-west-cookie-prefixes-05>.
- [51] WHATWG, “Fetch Standard,” <https://fetch.spec.whatwg.org/>.
- [52] —, “HTML - Living standard,” <https://html.spec.whatwg.org/>.

APPENDIX A WEB INVARIANTS

A. Cookies

1) *Integrity of `__Host-` Cookies:* In the following we give the complete Coq definition of the invariant defined in Section IV-A2. We encode the invariant as:

```
1 Definition HostInvariant (gb: Global) (evs: list Event) (st:
  State): Prop :=
2   ∀ rp corr pt sc ctx c_idx cookie _evs cname h,
3   Reachable gb evs st →
4   (
5     (
6       evs = (EvResponse rp corr :: _evs) ∧
7       (rp_hd_set_cookie (rp_headers rp)) = Some cookie ∧
8       (sc_name cookie) = (Host cname) ∧
9       url_host (rp_url rp) = Some h
10    ) ∨
11    (
12      is_script_in_dom_path gb (st_window st) pt sc ctx ∧
13      evs = (EvScriptSetCookie pt (DOMPath [] DOMTopLevel)
14        c_idx cookie :: _evs) ∧
15      (sc_name cookie) = (Host cname) ∧
16      url_host (wd_location ctx) = Some h
17    ) →
18    (sc_reg_domain cookie) = h.
```

A cookie can be set either via HTTP headers (lines 6-9) or via javascript (lines 12-15), however, when the name of the cookie has the `__Host-` prefix (lines 8 and 14), then the domain that registered the cookie must match (line 9) the URL of the response, or (line 15) the URL of the location of the window in which the script is running.

We can split the two cases in which the a cookie can be set and consider each case separately. When the cookie is set via HTTP headers the encoded invariant is:

```
1 Definition HostInvariantRP (gb: Global) (evs: list Event) (st:
  State): Prop :=
2   ∀ rp corr cookie _evs cname h,
3   Reachable gb evs st →
4   (* A response is setting a cookie *)
5   evs = (EvResponse rp corr :: _evs) →
6   (rp_hd_set_cookie (rp_headers rp)) = Some cookie →
7   (* The cookie prefix is __Host *)
8   (sc_name cookie) = (Host cname) →
9   (* The cookie has been registered by the domain of rp
10    *)
11   url_host (rp_url rp) = Some h →
12   (sc_reg_domain cookie) = h.
```

A counterexample of one of the two invariants is also a counterexample of the complete `HostInvariant`, since the complete invariant is equivalent to requiring both cases (`HostInvariantSC`, `HostInvariantRP`) to hold:

```
1 ∀ gb evs st,
2   HostInvariant gb evs st ↔ (HostInvariantRP gb evs st ∧
  HostInvariantSC gb evs st).
```

A proof of this equivalence is provided in [7].

2) *Confidentiality of `HttpOnly` cookies:* The `HttpOnly` attribute is designed to make cookies inaccessible to JavaScript both in read and write mode. This corresponds to the following invariant.

Invariant. *Scripts can only access the cookies without the `HttpOnly` attribute.*

We encode the invariant in our model as follows:

```
1 Definition HttpOnlyInvariant (gb: Global) (evs: list Event) (
  st: State): Prop :=
2   ∀ sc cm c_idx cookie,
3   Reachable gb evs st →
4   (* A script has access to the cookie cm *)
5   Scriptstate gb st sc (SOCookie c_idx cm) →
6   (* The cookie is not httponly *)
7   st_cookiejar st.[c_idx] = Some cookie →
8   cj_http_only cookie = false.
```

Where line 5 specifies that a script `sc` in the page have access to the cookie `cm` that is stored in the cookiejar at index `c_idx`; line 8 requires the cookie to have the `HttpOnly` flag set to false.

Attack. JavaScript is allowed to perform HTTP requests using various APIs, e.g., `XMLHttpRequest` and `fetch`, and programmatically access the contents of the response. In particular, the authors of [38] noticed that scripts could read the contents of the `Set-Cookie` header (through which cookies are set), thus violating the property that should be enforced by the `HttpOnly` flag.

When we configure our model to allow scripts to access the content of the `Set-Cookie` header, our toolchain produces a trace (shown in Figure 11) which shows that a script is able to access a `HttpOnly` cookie by reading the response headers of a response that contains a `Set-Cookie`. Modern

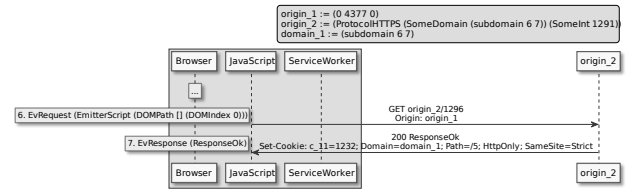


Fig. 11. `HttpOnly` Inconsistency

browsers have fixed the issue by preventing JS access to the `Set-Cookie` header contained in responses. We can configure our model so that `Set-Cookie` is a *forbidden header* [51] with `c_forbidden_headers (config gb) = true` and verify (up to a finite size) that the invariant holds.

B. Origin Header

The Origin header was proposed in [13] as a mechanism that websites can use to protect themselves against CSRF attacks. In particular, browsers populate this HTTP header with the origin that triggered the request being performed and Web servers should validate the header value to block undesired cross-origin requests.

1) *Authenticity of request initiator*: According to the proposal for the origin header [13], the header identifies the origin that initiated the request. If the browser is not able to determine the origin the header value should be null. So, when the Origin header value is different from null, no origin different from what is specified as the header value should be able to generate the request. This corresponds to the following Web invariant.

Invariant. If a request r includes the header $Origin: o$ (with $o \neq \text{null}$), then r was generated by origin o .

We encode the invariant in our model as follow:

```
1 Definition OriginInvariant (gb: Global) (evs: list Event) (st:
  State) : Prop :=
2   ∀ em rq corr _evs orghd orghd,
3   Reachable gb evs st →
4   (* Request with origin header orghd *)
5   evs = (EvRequest em rq corr :: _evs) →
6   rq_hd_origin (rq_headers rq) = Some orghd →
7   (* The source origin is equal to orghd *)
8   is_request_source gb st rq (Some orghd) →
9   orghd = orghd.
```

where the `is_request_source` predicate holds when `Some orghd` is the origin that generated the request `rq`. Note that the predicate needs to take into account redirections: the source of a redirected request is the origin of the server which performed the redirection.

Attack. In [8] the authors reported a vulnerability in the proposed CSRF protection caused by the fact that the header is preserved across cross-origin redirects. This way a POST request to the attacker can be redirected back to the honest server, that accepts it since the Origin header contains the expected value. When we configure our model to reflect the past state of the Web platform that was current at the time of [8] publication, we can rediscover an attack that breaks the invariant on the origin header. In particular, our toolchain produces the following counterexample: (i) The user visits a website hosted on `origin_1` and submits a form towards `origin_2`; (ii) The server on `origin_2` redirects the request back to `origin_1` using HTTP status code 307 to preserve HTTP method and request body; (iii) the browser follows the redirect and produces a new request towards `origin_1`; the request contains the header `Origin: origin_1`, since it preserved upon redirect. As a result, `origin_1` will accept the incoming request since the Origin header contains the expected value, thus voiding the CSRF protection. The output trace is shown in Figure 12.

Modern browsers tackle the issue by setting the header value to null in case of a cross-origin redirect, as dictated by the Fetch standard [51, §4.4]. We verify the security of the solution (up to a finite size) by disabling the origin header in cross origin redirects with the `c_origin_header_on_cross_origin_redirect (config gb) = false` configuration option.

C. Same Origin Policy and CORS

The Same-Origin Policy (SOP) is a security mechanism that restricts the interactions between documents loaded from

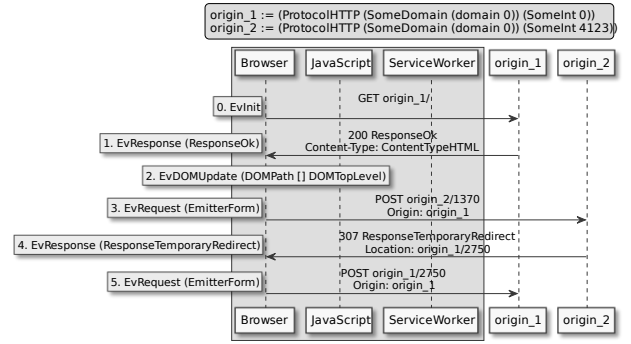


Fig. 12. Origin Header Inconsistency

different origins. The SOP can be relaxed for trusted websites using Cross-Origin Resource Sharing (CORS) [51, §3.2], a protocol that allows responses to specify the origins that are allowed to access their contents.

The CORS protocol distinguishes between simple and non-simple (or *preflighted*) requests depending on the request method, headers and contents [33]. In particular, simple requests use only the GET, HEAD, POST methods and are allowed to specify a limited sets of headers apart from the ones that are automatically added by the browser; preflighted requests are the requests that do not meet those conditions. Differently from simple requests which are safe to send cross-origin, preflighted requests require the browser to first issue a *pre-flight* request with the OPTIONS method to obtain the authorization to perform the actual request.

1) *Authorization of non-simple request (i)*: Following the specification for non-simple requests, we can define the relation between pre-flight and non-simple cross-origin requests as an invariant for the Same Origin Policy.

Invariant. A non-simple cross-origin request must be preceded by a pre-flight request.

We encode the invariant in our model as follows:

```
1 Definition SOPInvariant (gb: Global) (evs: list Event) (st:
  State) : Prop :=
2   ∀ rq corr em rest,
3   Reachable gb evs st →
4   evs = (EvRequest em rq corr :: rest) →
5   (* The request is a non-simple request *)
6   not (is_cors_simple_request rq) →
7   (* The request is cross origin *)
8   is_cross_origin_request (st_window st) rq →
9   (* There needs to be a preflight request *)
10  Exists (IsEvRequestCORSPreflight rq) rest.
```

where the `IsEvRequestCORSPreflight` holds when an event in the list is a pre-flight request.

Attack. Early drafts of the HTML5 standard added the possibility to use the HTTP methods PUT and DELETE in HTML forms. However, to avoid introducing vulnerabilities in existing websites, the specification requires to use this methods only on same-origin requests. The authors of [8] found that browsers were transparently following cross-origin redirects when using PUT and DELETE. When we configure our model to reflect the past state of the Web platform in which HTML

forms are allowed to use those methods, our toolchain is able to find a counterexample to the invariant (see Figure 13). In particular, when a same-origin PUT (step 3) is redirected to a different origin, the resulting request (step 5) is non-simple and cross-origin. Since requests generated by forms do not trigger a pre-flight, this request breaks the invariant on the Same origin Policy.

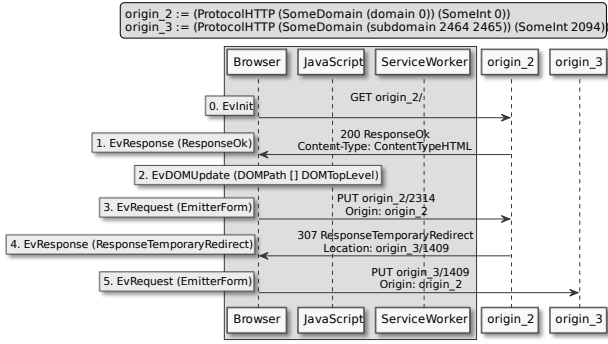


Fig. 13. Authorization of non-simple requests (i): cross-origin redirection of form-generated PUT request

The HTTP specification has been modified again to allow only HTTP methods GET and POST in form submissions [52, §4.10.18.6], so this problem does not affect modern browsers. We can disable early HTML5 form methods with `c_earlyhtml5_form_methods (config gb)= false` and verify (up to a finite size) that the invariant holds.

2) *Authorization of non-simple request (ii)*: The response to a pre-flight request declares, through the `Access-Control-Allow-Origin` header, which origins are allowed to perform the cross-origin request. Given that the pre-flight response authorizes an origin to perform potentially harmful cross-origin requests, we should enforce the following invariant.

Invariant. *The authorization to perform a non-simple request towards a certain origin o should come from o itself.*

That we encode in our model as follows:

```
1 Definition CORSInvariant (gb: Global) (evs: list Event) (st: State) : Prop :=
2   ∀ em rq corr scr_idx scr_pt rp rp_corr em_idx _evs,
3   Reachable gb evs st →
4   (* Non-simple request made by a script *)
5   evs = (EvRequest em rq corr :: _evs) →
6   em = EmitterScript scr_idx scr_pt ∧ (emitters gb).[em_idx] = em →
7   is_cross_origin_request (st_window st) rq →
8   not (is_cors_simple_request rq) →
9   (* Get CORS preflight response *)
10  is_cors_authorization_response gb st em_idx rq corr rp rp_corr →
11  (* The auth. comes from rq_url *)
12  origin_of_url (rq_url rq) = origin_of_url (rp_url rp).
```

Where `is_cors_authorization_response` (line 10) specifies that `rp` is the response to the CORS pre-flight request that is generated by the request `rq`; and line 12 requires that the origin the request `rq` is directed to must be the same one that generates the authorization response `rp`.

Attack. The original CORS draft allowed browsers to follow cross-origin redirects in responses to pre-flight requests [47]. When we configure our model to follow redirects for pre-flight response, our toolchain produces a counterexample, shown in Figure 14, in which after a redirection `origin_3` responds with `Access-Control-Allow-Origin: origin_1` to a request made by `origin_1` towards `origin_2`. Thus, a website running on `origin_2` containing open redirectors might redirect the pre-flight request to a server under the attacker's control, that by returning a CORS header allows the attacker to relax the Same Origin Policy for `origin_2`.

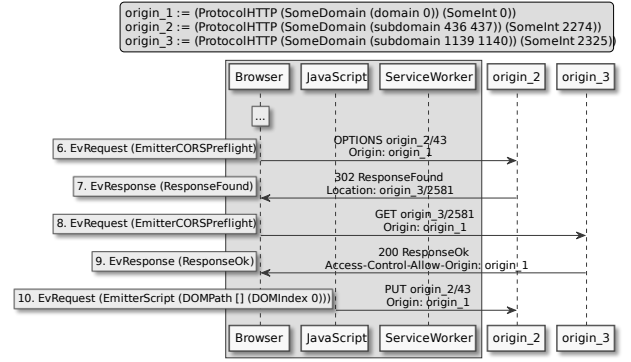


Fig. 14. Authorization of non-simple requests (ii): cross-origin redirection of pre-flight request

The most recent CORS specification (part of the Fetch Standard [51]) specifies that browser should ignore redirects in pre-flight responses. We can verify (up to a finite size) that the invariant holds by configuring our model to ignore pre-flight redirects with `c_redirect_preflight_requests (config gb)= false`.

APPENDIX B COMPILER

WebSpec includes a compiler that aims to find inhabitants of inductive types, a problem which is known to be undecidable for CIC, the logic of Coq [23]. To this end, the compiler translates terms in a fragment of CIC into CHC logic, i.e., first-order logic with fixed-points expressed in terms of Constrained Horn Clauses, hence discharging the undecidability of the problem to CHC solvers [28], [27]. In the following, we give an overview of how our compiler performs this translation.

A. Considered CIC Fragment

Contrary to related work [20], [19], our compiler does not perform a shallow embedding into *untyped* first-order logic, but instead performs a type-preserving translation into CHC logic, i.e., *typed* first-order logic with fixed-point. If, on the one hand, this allows us to leverage all the power of CHC solvers, this comes, on the other hand, at the price of restrictions on the fragment of the logic of Coq we consider.

The considered fragment of the logic of Coq we consider is CIC without dependent types (1), and where inductive type annotations and constructor arguments are restricted to ground

variables (2). We also require inductive type parameters to be instantiated when the compiler is called. Before discussing these limitations, note that the resulting logic is still extremely expressive as it contains System F_ω , the higher-order polymorphic lambda calculus. This also means that the inhabitation problem is still undecidable on this fragment [23].

The reason for the restriction on inductive type annotations and constructor arguments (2) is twofold. The first reason is that CHC solvers do not perform type equation resolution, and therefore introducing symbolic type variables is forbidden. It is possible to circumvent this issue by performing a shallow embedding of types, however this would likely come at a significant cost in resolution time. The second reason is similar to the first, but for functions. However in this case, we expect this restriction to be relaxed in the future thanks to recent progress in function synthesis [10], [36].

The restriction on dependent types (1) could also be circumvented by shallow embedding, but again at a high cost in resolution time. Instead, upcoming development of WebSpec aims to relax this restriction so that dependent types are allowed in inductive types. This relaxation will cover a significant number of practical cases, like the famous example where the type of an array includes a program expression giving the size of that array.

B. Compilation Pipeline

In order to translate the support fragment of CIC to CHC logic, our compiler performs the following steps:

Term-Type-Kind Hierarchy From a syntactic point of view, types cannot be distinguished from terms in CIC. Because CHC does not permit such intricacy, we have to build a strict term-type-kind stratified hierarchy [11], where kind are defined as $k := \text{Prop} \mid \text{Type} \mid k \rightarrow k$. This stratification is done by recursive exploration, starting from the inductive type on which the compiler is called, and following CIC typing rules² to deduce to which stratum each syntactic term belongs. We rely on Coq type-checking to ensure that connections between terms, types and kinds are sound. As a side effect, this stratification makes a clear distinction between types and proposition or between terms and proofs, which will ease subsequent steps.

Partial Application In CIC, any term can be partially applied. This include function of course, but also inductive types, constructors, or type definitions. Such flexibility is not allowed in CHC, and therefore all partial applications have to be removed. This is done by systematically performing η -expansion [21] on every term that could be applied.

Lambda Abstraction We also have to removed lambda abstractions, both those which are present in the original CIC terms and those which were introduced by η -expansion. To this end, we perform β -reduction wherever possible and remove remaining lambda-abstractions by lambda-lifting [29], [32].

Polymorphism and Higher-Order Thanks to the previous steps, all functions are now defined at top-level and totally

applied. Therefore we can now remove the use of polymorphism and higher-order simply by specialization: For every application of a function (resp. an inductive type) to a type or a function argument, we generate a specialized version of the function (resp. the inductive type) where the type or function parameter is replaced by the argument.

Constructor Constraints Constructors of inductive types in CIC can contain terms with arbitrary constraints, while CHC only supports simple algebraic datatypes. Therefore, we split every non-simple inductive type into a simple inductive type of kind `Type` and an inductive type of kind `Prop` which encapsulates these constraints.

Once these steps are done, the rest of the compilation is straightforward. Simple inductive types of kind `Type` are mapped to CHC algebraic datatypes, inductive types of kind `Prop` are mapped to relations, while CIC terms, types and proposition are mapped to CHC terms, sorts, and formulas.

²<https://coq.inria.fr/refman/language/cic.html>